

Instructions and submission rules

mohalya3@gmail.com [Switch account](#)



The name, email, and photo associated with your Google account will be recorded when you upload files and submit this form

* Indicates required question

Evaluation and scope

In this section, define the practical boundaries of the project.

You must clearly state:

- the minimum viable version of the project,
- how success will be evaluated,
- the major risks,
- and how your team plans to manage those risks.

This section is very important for approval.

Projects that are too broad, unevaluable, or highly risky without a mitigation plan may be returned for revision.



Minimum viable project scope *

What is the smallest complete version you promise to deliver? **(Very Important)**.

Our minimum viable product (MVP) is a fully containerized, microservice-based web application with six functional modules: Auth, Inventory, Order Processing, Warehouse Movement, Shipment Tracking, and Reporting. The MVP guarantees the completion of a full lifecycle workflow: a staff member can log in, add inventory, create an order that successfully deducts from that inventory, assign it to a warehouse zone, and mark it as dispatched. The MVP also includes a functioning CI/CD pipeline and a deployed cloud environment with basic Prometheus/Grafana logging.

Optional or stretch features

Real-time WebSocket notifications on the staff dashboard for order status changes.

Automated PDF generation for shipping labels and invoices.

Exportable CSV data drops for the Reporting module.

Implementation of distributed tracing (e.g., Jaeger) to track request flows across all six microservices.



How will you evaluate project success? (Very Important) *

State the technical and/or experimental metrics you will use.

Examples:

- accuracy / F1 / WER
- extraction quality
- latency
- recommendation quality
- scheduling objective score
- workflow completion
- security correctness
- availability / logging / demo stability

Workflow Completion: 100% success rate in end-to-end integration tests tracking an order from creation to shipment without data loss.

Logging / Observability Quality: All cross-service API errors must be successfully caught, centralized, and visible on our Grafana dashboard.

Latency: 95th percentile (p95) API response times remain under 300ms during our simulated load testing.

Availability: Doesn't crash under load.

Main project risks *

List the main technical, data, scope, or integration risks.

Integration and Contract Mismatch: With six distinct roles, there is a high risk of microservices failing to communicate properly due to mismatched API expectations (e.g., Order service sending data the Inventory service doesn't understand).

Cloud/Deployment Complexity: Setting up Docker, AWS/ECS, and CI/CD pipelines can become a bottleneck if left until the deadline.

Data Consistency: Managing state across distributed services (e.g., ensuring an item isn't double-sold while an order is processing) is a complex database risk.



Risk mitigation plan *

Communication between the team.

To mitigate deployment risks: The "Cloud & Observability Architect" will establish the Docker compose files and a basic "Hello World" CI/CD pipeline during Week 1, ensuring continuous integration from day one rather than attempting a onetime deployment at the deadline.

To mitigate data consistency risks: We will rely on robust PostgreSQL ACID transactions and explicit row-level locking for critical operations (like deducting stock levels) to prevent race conditions during high load.

[Back](#)[Next](#)

Page 10 of 13

[Clear form](#)

Never submit passwords through Google Forms.

This content is neither created nor endorsed by Google. - [Contact form owner](#) - [Terms of Service](#) - [Privacy Policy](#)

Does this form look suspicious? [Report](#)

Google Forms



